

Greedy Algorithm

คำชี้แจง: นักศึกษาต้องแสดงวิธีคิด ลำดับการเลือก และเหตุผลประกอบ การตอบเฉพาะผลลัพธ์สุดท้ายโดยไม่มีวิธีทำอาจไม่ได้คะแนนเต็ม

ข้อ 1 Activity Selection Greedy Choice & Trace

มหาวิทยาลัยต้องการจัดกิจกรรมในห้องประชุมหนึ่งห้อง โดยแต่ละกิจกรรมมีเวลาเริ่มต้นและเวลาสิ้นสุดดังนี้

Activity	Start	Finish
A	1	4
B	3	5
C	0	6
D	5	7
E	3	9
F	5	9
G	6	10
H	8	11
I	8	12
J	12	14

เป้าหมายคือเลือกกิจกรรมให้ได้ จำนวนมากที่สุด โดยกิจกรรมที่เลือกต้องไม่ทับเวลากัน

กำหนด Java class:

```
class Activity {  
    String name;  
    int start;  
    int finish;  
  
    Activity(String name, int start, int finish) {  
        this.name = name;  
        this.start = start;  
        this.finish = finish;  
    }  
}
```

1.1 จงตอบคำถามต่อไปนี้

1. Objective ของปัญหานี้คืออะไร

ตอบ: เลือกกิจกรรมให้ได้จำนวนมากที่สุด โดยกิจกรรมที่เลือกต้องไม่ทับเวลากัน

2. Greedy Choice ที่เหมาะสมคืออะไร

ตอบ: เลือกกิจกรรมที่มีเวลาเสร็จ (**Finish**) เร็วที่สุดก่อน เพื่อเหลือเวลาให้กิจกรรมถัดไปมากที่สุด

3. Feasibility Condition สำหรับการเลือกกิจกรรมถัดไปคืออะไร

ตอบ: กิจกรรมถัดไปต้องมี **Start $\geq$ Finish** ของกิจกรรมที่เลือกไว้ก่อนหน้านี้ (**Start $\geq$ lastFinish**)

1.2 ใช้ Greedy Strategy แก้ปัญหา โดยแสดงลำดับกิจกรรมที่พิจารณาและกิจกรรมที่ถูกเลือกทุกขั้นตอน โดยมีตัวอย่างรูปแบบคำตอบ

Step	Activity	Start	Finish	Feasible?	Decision
1	A	1	4	ใช่	เลือก A (กิจกรรมแรก)
2	B	3	5	ไม่ใช่	ไม่เลือก เพราะ $3 < 4$
3	C	0	6	ไม่ใช่	ไม่เลือก เพราะ $0 < 4$
4	D	5	7	ใช่	เลือก D เพราะ $5 \geq 4$
5	E	3	9	ไม่ใช่	ไม่เลือก เพราะ $3 < 7$
6	F	5	9	ไม่ใช่	ไม่เลือก เพราะ $5 < 7$
7	G	6	10	ไม่ใช่	ไม่เลือก เพราะ $6 < 7$
8	H	8	11	ใช่	เลือก H เพราะ $8 \geq 7$
9	I	8	12	ไม่ใช่	ไม่เลือก เพราะ $8 < 11$
10	J	12	14	ใช่	เลือก J เพราะ $12 \geq 11$

จากนั้นระบุลำดับกิจกรรมที่ถูกเลือก

Selected Activities = A, D, H, J

และจำนวนกิจกรรมทั้งหมด

Number of Activities = 4

1.3 เลือกและเรียงลำดับคำสั่ง Java

กำหนดรายการคำสั่ง Java ต่อไปนี้ โดยมีทั้งคำสั่งที่ จำเป็น และคำสั่งที่ ไม่ควรใช้

จงเลือกเฉพาะคำสั่งที่จำเป็น และเรียงลำดับเพื่อสร้าง Greedy Algorithm สำหรับ Activity Selection ให้ถูกต้อง

จากนั้นเขียนเป็น **Java code** ที่สมบูรณ์

รายการคำสั่ง

A. `activities.sort(Comparator.comparingInt(a -> a.finish));`

B. `activities.sort(Comparator.comparingInt(a -> a.start));`

C. `List<Activity> selected = new ArrayList<>();`

D. `int lastFinish = -1;`

E. `int lastFinish = Integer.MAX_VALUE;`

F. `for (Activity a : activities) {`

G. `if (a.start  $\geq$  lastFinish) {`

H. `if (a.finish  $\geq$  lastFinish) {`

I. `selected.add(a);`

J. lastFinish = a.finish;

K. lastFinish = a.start;

L. }

M. Collections.reverse(activities);

N. selected.clear();

คำสั่งที่เลือกและเรียงลำดับ: $A \rightarrow C \rightarrow D \rightarrow F \rightarrow G \rightarrow I \rightarrow J \rightarrow L$

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Activity {
    String name;
    int start;
    int finish;

    Activity(String name, int start, int finish) {
        this.name = name;
        this.start = start;
        this.finish = finish;
    }
}

public class ActivitySelection {
    public static void main(String[] args) {
        List<Activity> activities = new ArrayList<>();

        activities.add(new Activity("A", 1, 4));
        activities.add(new Activity("B", 3, 5));
        activities.add(new Activity("C", 0, 6));
        activities.add(new Activity("D", 5, 7));
        activities.add(new Activity("E", 3, 9));
        activities.add(new Activity("F", 5, 9));
        activities.add(new Activity("G", 6, 10));
        activities.add(new Activity("H", 8, 11));
        activities.add(new Activity("I", 8, 12));
        activities.add(new Activity("J", 12, 14));

        activities.sort(Comparator.comparingInt(a -> a.finish));

        List<Activity> selected = new ArrayList<>();
        int lastFinish = -1;

        for (Activity a : activities) {
            if (a.start >= lastFinish) {
                selected.add(a);
                lastFinish = a.finish;
            }
        }

        System.out.println("Selected Activities:");
        for (Activity a : selected) {
            System.out.println(a.name + " (" + a.start + ", " + a.finish + ")");
        }
        System.out.println("Number of Activities = " + selected.size());
    }
}
```

1.4 จงวิเคราะห์ Time Complexity ของ Algorithm เมื่อข้อมูลกิจกรรม ยังไม่ได้เรียงลำดับ
ให้อธิบาย

1. ขั้นตอน Sorting มี Complexity เท่าใด

ตอบ: $O(n \log n)$

2. ขั้นตอนตรวจสอบกิจกรรมมี Complexity เท่าใด

ตอบ: $O(n)$

3. Complexity รวมเป็นเท่าใด

ตอบ: $O(n \log n)$ เพราะการ **Sorting** ใช้เวลามากกว่าการตรวจสอบ